THLPro — Guia de Instalação

Sistema profissional de análise de opções em tempo real (substitui a planilha THLPro.xlsx). Este guia cobre instalação em desenvolvimento e em produção.

> Resumo das peças: **backend** (Python/FastAPI), **frontend** (React/Vite), **cliente Windows** (Python/.exe que lê o Profit via RTD), **PostgreSQL + nginx** via Docker.

---

1. Pré-requisitos

- **Servidor:** Linux com Docker e Docker Compose (v2), ou Python 3.12 + PostgreSQL 16 para instalação manual.
- **Máquina de operação:** Windows 10/11 64-bit com **Profit Pro (Nelogica)** instalado e o servidor RTD ativo (ver `docs/MANUAL_PROFIT_RTD.md`).
- **Navegadores:** Chrome/Edge (recomendados) — a tela funciona em desktop, notebook, tablet e telas menores.

---

2. Instalação rápida com Docker (produção recomendado)

```
# 1. Obter o projeto
git clone <seu-repositorio>/thlpro.git
cd thlpro

# 2. Configurar variáveis de ambiente
cp .env.example .env
#   → EDITE .env: troque POSTGRES_PASSWORD, SECRET_KEY e ADMIN_PASSWORD (obrigatório!)

# 3. Subir os serviços (db, api, web)
docker compose up -d --build

# 4. Acompanhar a inicialização (api roda migrações + seed automaticamente)
docker compose logs -f api

# 5. Acessar
#   http://localhost                → frontend (tela de login)
#   http://localhost/health         → status da API
#   http://localhost/docs           → Swagger/OpenAPI
# Credenciais iniciais: ADMIN_EMAIL / ADMIN_PASSWORD do .env
```

A API só começa a atender depois que o banco estiver saudável (healthcheck) e as migrações+seed terminarem.

2.1 Com Redis (opcional, escala horizontal)

```
docker compose --profile with-redis up -d --build
# e no .env: USE_REDIS=true
```

3. Instalação manual (desenvolvimento)

3.1 Backend

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env        # ajuste DATABASE_URL e SECRET_KEY

# PostgreSQL local
sudo -u postgres createdb thlpro
sudo -u postgres createdb thlpro_test   # só para testes

# Migrações + seed (admin + ativos BOVA11 e ABEV3)
alembic upgrade head
python -m scripts.seed

# API
uvicorn app.main:app --host 0.0.0.0 --port 8000
#   Swagger: http://localhost:8000/docs
```

3.2 Frontend

```
cd frontend
npm install
npm run dev        # http://localhost:5173 (proxy /api e /ws para o backend local)
npm run build      # build de produção em dist/
```

3.3 Cliente Windows (na máquina do Profit)

Ver `docs/MANUAL_PROFIT_RTD.md` (configuração do Profit/RTD) e `client/README.md` (build do .exe com PyInstaller e instalador Inno Setup). Resumo:

```
cd client
pip install -r requirements.txt
python build_win.bat          :: gera dist/THLProClient.exe (PyInstaller)
:: Instalar com Instalador THLPro.iss (Inno Setup) — gera o instalador .exe
```

4. Funcionamento do fluxo de dados

1. O **cliente Windows** lê do Profit (RTD `rtdtrading.rtdserver`) o último preço (`ULT`), BID (`OCP`), ASK (`OVD`), vencimento (`VEN`) e strike (`PEX`) das séries ativas.
2. Ele envia as cotações para o backend (`POST /clients/quotes`), que valida (timestamp, anti-roll-back, regra de prioridade entre clientes) e grava no PostgreSQL.
3. O backend **publica** a atualização via WebSocket (`/ws/market-data`) para **todos os usuários conectados** — cada tela atualiza sem recarregar, mantendo suas próprias preferências (ativo, strike, tema).
4. A sincronização das **séries de opções** (códigos, strikes, vencimentos) é feita pelo arquivo oficial da B3, manualmente ou em intervalos programados: `POST /options/synchronize` (admin).

5. Produção — HTTPS

Com nginx exposto (portas 80/443 no compose), habilite TLS:

```
docker compose cp nginx/certs.crt web:/etc/nginx/conf.d/  # ou use certbot:
docker compose exec web sh -c "apk add certbot certbot-nginx"
```

Ou, mais simples: coloque o **nginx do host** (ou um reverse proxy como Caddy/Traefik) na frente e aponte para `localhost:80` do compose — recomendado para subdomínios com certificados automáticos.

Importante: em produção, sirva por HTTPS, troque `SECRET_KEY` /senhas e restrinja `CORS_ORIGINS`.

6. Backup do banco

```
# Dentro do container:
docker compose exec db pg_dump -U thlpro thlpro -Fc > backup_$(date +%Y%m%d_%H%M).dump

# Restaurar:
docker compose exec -T db pg_restore -U thlpro -d thlpro --clean < backup.dump
```

Sugestão: agende o dump diário em crontab (horário sem movimento — após 22h e antes das 9h da B3) e retenha 30 dias.

7. Atualização do sistema

```
cd thlpro
git pull
docker compose up -d --build      # migrações rodam automaticamente na subida
docker compose logs -f api        # confirme "alembic upgrade head" sem erros
```

8. Telas e perfis

- **Usuário comum:** login, tela principal (ativos, strikes, CALL/PUT, gráficos), preferências individuais.
- **Administrador:** usuários, ativos, clientes/diagnóstico, logs, sincronização B3, configurações do sistema (tolerância de strike, tempo de validade do dado, modo simulação).

Documentação de instalação — THLPro. Consulte também `docs/API.md`, `docs/MANUAL_USUARIO.md` e `docs/MANUAL_PROFIT_RTD.md`.